

实验二 任务 spawning 与调度

一、实验目的

1. 掌握任务调度的基本原理和实现方法。
2. 实现基于 Rust 的任务 spawning 与调度，学习 Rust 异步编程和并发机制。

二、实验原理

2.1 任务调度

操作系统的任务调度是管理进程执行的核心部分。任务调度的目标是合理分配处理器时间，确保系统稳定高效地运行。现代操作系统通常支持多任务处理，每个任务（或进程）会根据一定的策略轮流占用 CPU 时间。这种调度策略常见的有：轮转调度、优先级调度等。

在现代操作系统中，为了提高任务并发性和响应速度，异步编程（如 `async/await`）被广泛应用。异步任务不需要一直占用 CPU，而是通过“挂起”自己，在任务执行的过程中等待外部事件或资源的完成后再继续执行。

2.2 `async/await` 与任务调度

在 Rust 中，`async` 关键字用于声明异步函数，`await` 用于等待异步操作的结果。Rust 的异步编程模型基于“任务”的概念，这些任务可以被调度到执行队列中，直到其能够继续执行。为此，我们需要实现一个轻量级的异步任务调度器。

在实现异步任务调度时，任务的“生成”（task spawning）和“重生成”是两个重要的概念：

- 任务生成：创建一个新的异步任务并将其加入到任务队列中。
- 任务重生成：某些情况下，任务可能需要重新加入队列进行重新调度。

2.3 实验重点

Spawner 是一种特殊的结构，它能在 executor 运行时创建新的任务并放入任务队列中。

本实验基于 Rust 编程语言实现：

- **Executor**：任务调度器，负责管理任务的队列以及任务的执行。
- **Task**：表示一个异步任务，包含任务的 ID 和状态。
- **Spawner**：用于创建和提交任务到调度器中。
- **TaskWaker**：为每个任务生成一个 Waker，使得任务能够被唤醒并重新调度。

三、实验内容

1. 实现一个任务结构，包含任务名称（或标识符）。
2. 创建 Spawning 结构，接受一个任务并将其加入到 new_tasks 队列中。
3. 通过例子进行测试，实现 Spawner 在运行时的操作。

四、实验步骤

4.1 代码结构概览

- Task: 定义任务结构体，包括任务 ID、异步任务和优先级。
- Executor: 任务执行器，管理任务队列和任务调度。
- TaskWaker: 自定义 Waker，用于唤醒任务。
- Spawner: 自定义 Spawner，用于新建任务。

4.2 核心代码说明

src/task/executor.rs

```
use super::{Task, TaskId};
use alloc::{collections::BTreeMap, sync::Arc, task::Wake};
use core::task::{Context, Poll, Waker};
use crossbeam_queue::ArrayQueue;
use crossbeam_queue::SegQueue;
pub struct Executor {
    tasks: BTreeMap<TaskId, Task>,
    task_queue: Arc<ArrayQueue<TaskId>>,
    waker_cache: BTreeMap<TaskId, Waker>,
    new_tasks: Arc<SegQueue<Task>>,
}
pub struct Spawner {
    new_tasks: Arc<SegQueue<Task>>,
}
impl Spawner {
    pub fn spawn(&self, task: Task) {
        self.new_tasks.push(task);
    }
}

impl Executor {
    pub fn new() -> Self {
        Executor {
            tasks: BTreeMap::new(),
            task_queue: Arc::new(ArrayQueue::new(100)),
            waker_cache: BTreeMap::new(),
        }
    }
}
```

```

        new_tasks: Arc::new(SegQueue::new()),
    }
}
pub fn spawner(&self) -> Spawner {
    Spawner {
        new_tasks: self.new_tasks.clone(),
    }
}
pub fn spawn(&mut self, task: Task) {
    let task_id = task.id;
    if self.tasks.insert(task.id, task).is_some() {
        panic!("task with same ID already in tasks");
    }
    self.task_queue.push(task_id).expect("queue full");
}

pub fn run(&mut self) -> ! {
    loop {
        // 将新任务加入任务列表
        while let Some(task) = self.new_tasks.pop() {
            let task_id = task.id;
            if self.tasks.insert(task_id, task).is_some() {
                panic!("task with same ID already in tasks");
            }
            self.task_queue.push(task_id).expect("queue full");
        }
        self.run_ready_tasks();
        self.sleep_if_idle();
    }
}

fn run_ready_tasks(&mut self) {
    // destructure `self` to avoid borrow checker errors
    let Self {
        tasks,
        task_queue,
        waker_cache,
        new_tasks,
    } = self;

    while let Some(task_id) = task_queue.pop() {
        let task = match tasks.get_mut(&task_id) {
            Some(task) => task,
            None => continue, // task no longer exists
        };
    }
}

```

```

        };
        let waker = waker_cache
            .entry(task_id)
            .or_insert_with(|| TaskWaker::new(task_id,
task_queue.clone()));
        let mut context = Context::from_waker(waker);
        match task.poll(&mut context) {
            Poll::Ready(()) => {
                // task done -> remove it and its cached waker
                tasks.remove(&task_id);
                waker_cache.remove(&task_id);
            }
            Poll::Pending => {}
        }
    }
}

fn sleep_if_idle(&self) {
    use x86_64::instructions::interrupts::{self,
enable_and_hlt};

    interrupts::disable();
    if self.task_queue.is_empty() {
        enable_and_hlt();
    } else {
        interrupts::enable();
    }
}

struct TaskWaker {
    task_id: TaskId,
    task_queue: Arc<ArrayQueue<TaskId>>,
}

impl TaskWaker {
    fn new(task_id: TaskId, task_queue: Arc<ArrayQueue<TaskId>>) ->
Waker {
        Waker::from(Arc::new(TaskWaker {
            task_id,
            task_queue,
        }))
    }
}

```

```

        fn wake_task(&self) {
            self.task_queue.push(self.task_id).expect("task_queue
full");
        }
    }

    impl Wake for TaskWaker {
        fn wake(self: Arc<Self>) {
            self.wake_task();
        }

        fn wake_by_ref(self: &Arc<Self>) {
            self.wake_task();
        }
    }
}

```

4.3 实验测试方法

任务创建与调度测试:

- 创建多个任务。包括 Executor 和 Spawner
- 调用 Executor::run 方法，验证 Executor 任务是否优先执行。
- 在任务调用 Spawner，验证任务是否被放入队列并执行。

```

#![no_std]
#![no_main]
#![feature(custom_test_frameworks)]
#![test_runner(blog_os::test_runner)]
#![reexport_test_harness_main = "test_main"]

extern crate alloc;

use blog_os::println;
use blog_os::task::{executor::Executor, keyboard, Task};
use bootloader::{entry_point, BootInfo};
use futures_util::task::Spawn;
use core::panic::PanicInfo;

entry_point!(kernel_main);

fn kernel_main(boot_info: &'static BootInfo) -> ! {
    use blog_os::allocator;
    use blog_os::memory::{self, BootInfoFrameAllocator};
    use x86_64::VirtAddr;

```

```

println!("Hello World{}", "!");
blog_os::init();

let phys_mem_offset =
VirtAddr::new(boot_info.physical_memory_offset);
let mut mapper = unsafe { memory::init(phys_mem_offset) };
let mut frame_allocator = unsafe
{ BootInfoFrameAllocator::init(&boot_info.memory_map) };

allocator::init_heap(&mut mapper, &mut
frame_allocator).expect("heap initialization failed");

#[cfg(test)]
test_main();

let mut executor = Executor::new();
executor.spawn(Task::new(keyboard::print_keypresses()));
let spawner = executor.spawner();
spawner.spawn(Task::new(example_task()));
executor.run();
}

/// This function is called on panic.
#[cfg(not(test))]
#[panic_handler]
fn panic(info: &PanicInfo) -> ! {
    println!("{}", info);
    blog_os::hlt_loop();
}

#[cfg(test)]
#[panic_handler]
fn panic(info: &PanicInfo) -> ! {
    blog_os::test_panic_handler(info)
}

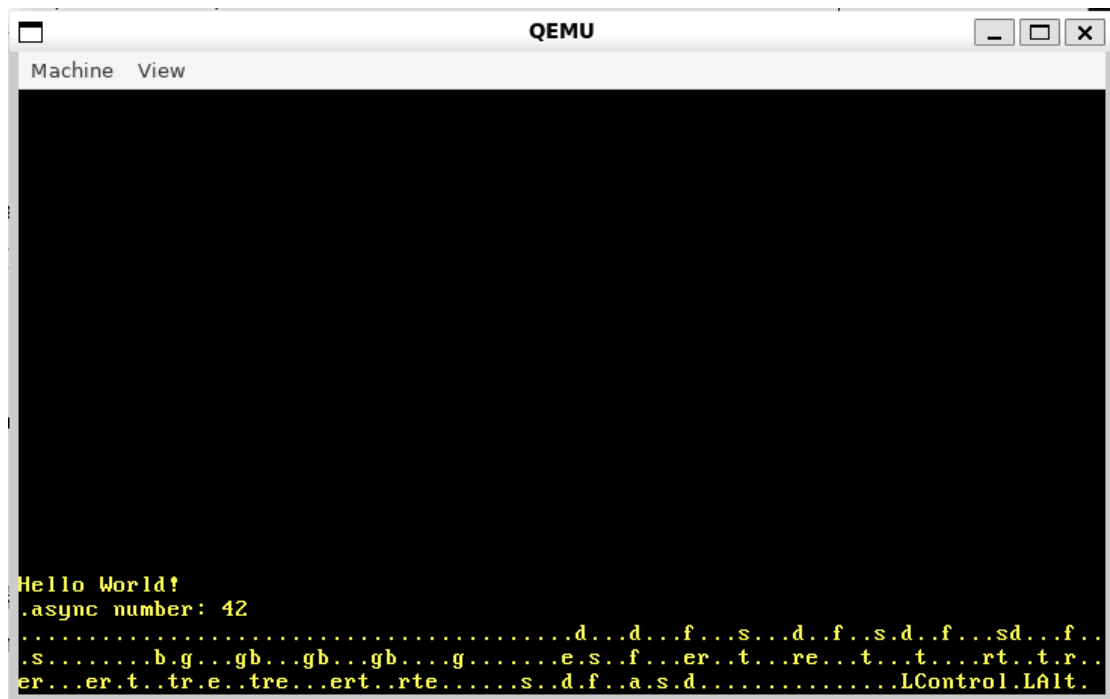
async fn async_number() -> u32 {
    42
}

async fn example_task() {
    let number = async_number().await;
    println!("async number: {}", number);
}

```

```
#[test_case]
fn trivial_assertion() {
    assert_eq!(1, 1);
}
```

4.4 运行结果



可以看到 Executor 和 Spawner 的任务都执行了，证明 Spawner 有效。

五、与现有实验相比的创新点

5.1 完善性

与传统调度算法相比，本实验加入了 `Spwaner`，能够在 `Executor` 正在运行无法新建任务时新建任务。使用 `Rust` 的 `async/await` 特性实现异步任务调度，符合现代编程趋势。通过本实验，可以深入理解任务调度的原理，并提升对 `Rust` 异步编程模型的理解和实践能力。

5.2 创新点

- **异步任务调度:**
 - 现有的实验可能只涉及简单的任务调度，本实验引入了 Rust 中的 `async/await` 机制，通过轻量级的任务调度器实现了异步任务的生成、挂起与重调度。
- **任务重生成机制:**
 - 通过设计 `TaskWaker` 结构体并实现 `Wake trait`，实现了任务的唤醒机制和重生成功能。这对于理解和实现高效的任务调度非常重要。

- **CPU 空闲时休眠机制：**
 - 在任务队列为空时，通过硬件中断指令让 CPU 进入低功耗休眠状态，节省系统资源，这对于嵌入式系统或资源受限的环境尤为重要。
- **基于队列的任务管理：**
 - 通过 ArrayQueue 和 SegQueue 等数据结构管理任务队列，确保任务调度的高效性和线程安全性。